



Tarea 5.1: Consultas Avanzadas, Stored Procedures y Triggers en SQL Server

Universidad Autonoma de Santo Domingo (UASD)

Facultad de Ciencias - Escuela de Informatica

Maestria en Ciencia de Datos e Inteligencia Artificial

Asignatura: Base de Datos I - INF-8236-C2

Unidad 5: Administracion, Optimizacion y Tendencias en Bases de Datos

Grupo: 7

Integrantes: Manicaotex Mueses, Roberto Byas de la Cruz, Victor Perez Padilla y Marlenis

Judith Concepcion Cuevas

Participante que entrega esta version: Marlenis Judith Concepcion Cuevas

Tutora: Mtra. Rosmery Alberto M.

Fecha: 19 de abril de 2026

Tabla de Contenido

- Introduccion
- Ejercicio 1. Analisis de planes de ejecucion
- Ejercicio 2. Indices y optimizacion del rendimiento
- Ejercicio 3. Estadisticas en SQL Server
- Ejercicio 4. Concepto ACID y manejo de transacciones
- Ejercicio 5. Uso de ROLLBACK y SAVEPOINT
- Ejercicio 6. Concurrencia y bloqueos
- Ejercicio 7. Niveles de aislamiento
- Ejercicio 8. Stored procedures
- Ejercicio 9. Funciones en SQL Server
- Ejercicio 10. Triggers
- Ejercicio 11. Vista materializada
- Ejercicio 12. Seguridad: usuarios, roles y permisos
- Conclusiones
- Referencias

Introduccion

El presente informe desarrolla la Tarea 5.1 correspondiente a la Unidad 5 de la asignatura Base de Datos I. La actividad requiere trabajar sobre una base de datos de nomina en SQL Server, aplicando conceptos de optimizacion, planes de ejecucion, indices, estadisticas, propiedades ACID, manejo de transacciones, concurrencia, niveles de aislamiento, procedimientos almacenados, funciones, triggers, vistas materializadas y seguridad.

Para responder la actividad se construyo la base de datos `DBNomina`, se completaron los objetos faltantes del modelo fisico, se agregaron datos de prueba coherentes con el contexto academico y se implementaron procesos de auditoria, historico y cache para demostrar la automatizacion mediante SQL Server. Ademas, se aplicaron reglas reales de Republica Dominicana para el calculo de AFP, ARS e ISR, con el fin de que las transacciones de nomina tengan una logica financiera mas realista.

Archivos Entregados

- `Marlenis-Concepcion-INF8236-Tarea-5.1-DBNomina.sql`
- `Marlenis-Concepcion-INF8236-Tarea-5.1-Respuestas-y-Evidencias.pdf`
- `Marlenis-Concepcion-INF8236-Tarea-5.1-Presentacion.pptx`

Consideraciones Tecnicas Previas

- Se agrego la tabla `TipoPeriodoNomina` porque el enunciado la menciona, pero no la crea.
- Se agrego la columna `TransaccionNomina\_Monto` porque el ejercicio 3 consulta esa columna de forma explicita.
- La vista materializada se implemento como `indexed view`, que es el equivalente practico en SQL Server.

- Para demostrar el trigger asociado a `SP\_Consultar\_NominaEmpleado`, se creo la tabla `NominaEmpleadoConsultaCache`.
- Se crearon tablas auxiliares de auditoria e historico para que los triggers tengan evidencia visible.
- Los porcentajes usados en los calculos fueron: AFP empleado 2.87%, ARS empleado 3.04% y escala de ISR 2026 de DGII.

Ejercicio 1. Analisis de Planes de Ejecucion

Objetivo

Analizar el plan de ejecucion de una consulta que recupere las transacciones de nomina de un empleado especifico.

Script Ejecutado

```
SELECT *
FROM dbo.TransaccionNomina
WHERE Empleado_ID = 3;
```

Explicacion Detallada

Esta consulta solicita todas las columnas de la tabla `TransaccionNomina`, pero solamente para las filas cuyo `Empleado\_ID` sea igual a `3`. En terminos del motor de SQL Server, esto significa que el optimizador debe localizar todas las filas relacionadas con ese empleado.

Antes de crear el indice sobre `Empleado\_ID`, el motor no dispone de una estructura especializada para buscar directamente por esa columna. Por esa razon, el comportamiento esperado del plan de ejecucion es revisar la estructura clustered principal de la tabla y recorrer filas hasta encontrar las coincidencias. En un escenario pequeno la diferencia puede parecer menor, pero conceptualmente el costo crece cuando la tabla aumenta de tamano.

Resultado Esperado de la Corrida

ID	Numero	Periodo	Monto Bruto	AFP	ARS	ISR	Sueldo Neto
3	1003	202601	78000.00	2238.60	2371.20	6930.49	66459.71
6	1006	202602	80000.00	2296.00	2432.00	7400.94	67871.06

Respuestas

1. El tipo de operacion esperado antes de optimizar es `Clustered Index Scan`.
2. Antes de crear el indice no se observa `Index Seek`, sino un `Scan`.
3. El costo estimado baja despues de crear el indice; si la consulta se ejecuta sola, SSMS suele mostrar el 100% del lote como costo relativo, por lo que el punto realmente importante es la mejora comparativa del plan.

Ejercicio 2. Indices y Optimizacion del Rendimiento

Objetivo

Crear un indice no agrupado sobre `Empleado\_ID` para mejorar las consultas por empleado.

Script Ejecutado

```
CREATE NONCLUSTERED INDEX IX_TransaccionNomina_Empleado_ID
ON dbo.TransaccionNomina (Empleado_ID);

SELECT *
FROM dbo.TransaccionNomina
WHERE Empleado_ID = 3;
```

Explicacion Detallada

La primera sentencia crea un indice no agrupado llamado `IX\_TransaccionNomina\_Empleado\_ID`. Ese indice guarda los valores de la columna `Empleado\_ID` de forma ordenada y con referencias a las filas de la tabla base. Gracias a eso, la segunda consulta puede ir directamente a la porcion del indice que contiene el valor `3`, en vez de revisar una gran cantidad de registros.

En terminos de optimizacion, el cambio importante no es solo de velocidad, sino de estrategia. Sin indice, el motor lee mas informacion de la necesaria. Con indice, el motor enfoca la busqueda de manera puntual. Si se usa `SELECT \*`, aun puede aparecer un `Key Lookup` para completar columnas no cubiertas por el indice, pero la mejora principal sigue existiendo.

Comparacion Antes y Despues

```
Antes    : Clustered Index Scan
Despues  : NonClustered Index Seek
```

Respuestas

1. La diferencia observable es que el plan cambia de un `Scan` a un `Seek`.
2. Si, el rendimiento mejora porque se reduce la lectura de paginas y el acceso a los datos es mas dirigido.

Ejercicio 3. Estadísticas en SQL Server

Objetivo

Actualizar las estadísticas de la tabla `TransaccionNomina` y observar el impacto en el optimizador.

Script Ejecutado

```
UPDATE STATISTICS dbo.TransaccionNomina;  
  
SELECT *  
FROM dbo.TransaccionNomina  
WHERE TransaccionNomina_Monto > 20000;
```

Explicacion Detallada

La sentencia `UPDATE STATISTICS` le indica a SQL Server que vuelva a calcular la distribución de datos de la tabla. Las estadísticas son fundamentales porque informan al optimizador cuantas filas es probable que cumplan una condición. Esa estimación influye directamente en la selección del plan de ejecución.

La segunda consulta filtra todas las transacciones con monto mayor a RD\$20,000. El optimizador necesita saber si la condición devolverá pocas filas, muchas filas o casi toda la tabla. Si la estimación es mala, puede elegir un plan ineficiente. Si las estadísticas están actualizadas, la decisión será más acertada.

Resultado Esperado de la Corrida

Numero	Empleado	Monto Bruto	Sueldo Neto
1001	Marlenis Judith Concepcion Cuevas	42000.00	38792.88
1002	Roberto Byas De la Cruz	55000.00	49189.83
1003	Victor Perez Padilla	78000.00	66459.71
1004	Manicaotex Mueses	120000.00	96098.06
1005	Rosmery Alberto Martinez	250000.00	188240.97
1006	Victor Perez Padilla	80000.00	67871.06
1007	Ana Martinez	32000.00	30108.80
1008	Marlenis Judith Concepcion Cuevas	42000.00	38792.88
1009	Roberto Byas De la Cruz	55000.00	49189.83

Respuesta

Las estadísticas son importantes porque ayudan al optimizador a estimar cardinalidad, seleccionar mejores planes de ejecución y evitar decisiones costosas o innecesarias.

Ejercicio 4. Concepto ACID y Manejo de Transacciones

Objetivo

Insertar una nueva transaccion de nomina bajo control transaccional y confirmarla con `COMMIT`.

Script Ejecutado

```
BEGIN TRY
    BEGIN TRAN;

    INSERT INTO dbo.TransaccionNomina
    (
        Compania_Codigo,
        TransaccionNomina_Numero,
        TransaccionNomina_Fecha,
        Empleado_ID,
        TipoNomina_Codigo,
        PeriodoNomina_Codigo,
        TransaccionNomina_Comentario,
        TransaccionNomina_Monto,
        TransaccionNomina_Estado
    )
    VALUES
    ('UAS', 1007, '2026-03-31', 6, 'FIX', '202603',
    'Pago mensual confirmado con COMMIT', 32000.00, 'Pendiente');

    COMMIT TRAN;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRAN;
    THROW;
END CATCH;
```

Explicacion Detallada

La transaccion se inicia con ``BEGIN TRAN``, lo que agrupa la operacion ``INSERT`` dentro de una unidad de trabajo. Si todo sale bien, el bloque ejecuta ``COMMIT`` y el cambio queda confirmado en la base de datos. Si ocurre un error, el bloque ``CATCH`` revierte la operacion mediante ``ROLLBACK``.

Esto demuestra la propiedad de atomicidad: la operacion debe completarse por entero o no aplicarse en absoluto. Tambien demuestra la durabilidad cuando el ``COMMIT`` se concreta.

Resultado Esperado de la Corrida

ID	Numero	Empleado	Monto Bruto	AFP	ARS	ISR	Sueldo Neto
7	1007	Ana Martinez	32000.00	918.40	972.80	0.00	30108.80

Respuestas

1. Si ocurre un error antes del ``COMMIT``, los cambios no deben permanecer porque se ejecuta ``ROLLBACK``.
2. La propiedad ACID que garantiza permanencia es ``Durability``.

Ejercicio 5. Uso de ROLLBACK y SAVEPOINT

Objetivo

Simular una transaccion con dos inserciones, un punto parcial y una actualizacion que se revierte.

Script Ejecutado

```

BEGIN TRY
    BEGIN TRAN;

    INSERT INTO dbo.TransaccionNomina
    (
        Compania_Codigo, TransaccionNomina_Numero, TransaccionNomina_Fecha,
        Empleado_ID, TipoNomina_Codigo, PeriodoNomina_Codigo,
        TransaccionNomina_Comentario, TransaccionNomina_Monto, TransaccionNomina_Estado
    )
    VALUES
    ('UAS', 1008, '2026-03-31', 1, 'FIX', '202603',
     'Transaccion para prueba SAVEPOINT 1', 42000.00, 'Pendiente'),
    ('UAS', 1009, '2026-03-31', 2, 'FIX', '202603',
     'Transaccion para prueba SAVEPOINT 2', 55000.00, 'Pendiente');

    SAVE TRAN PuntoParcial;

    UPDATE dbo.TransaccionNomina
    SET TransaccionNomina_Comentario = 'Actualizacion revertida por SAVEPOINT'
    WHERE TransaccionNomina_Numero = 1009;

    ROLLBACK TRAN PuntoParcial;
    COMMIT TRAN;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRAN;
    THROW;
END CATCH;

```

Explicacion Detallada

En este ejercicio la transaccion comienza insertando dos registros. Luego se marca un punto de control con `SAVE TRAN PuntoParcial`. A partir de ese momento se realiza una actualizacion. Cuando se ejecuta `ROLLBACK TRAN PuntoParcial`, SQL Server deshace solo lo ocurrido despues del savepoint, pero conserva lo anterior.

Este comportamiento es muy util en procesos complejos, porque permite corregir una parte del

flujo sin destruir toda la operacion completa.

Resultado Esperado de la Corrida

Numero	Comentario final
1008	Transaccion para prueba SAVEPOINT 1
1009	Transaccion para prueba SAVEPOINT 2

Respuestas

1. Permanecen los dos registros insertados, porque fueron creados antes del savepoint.
2. La ventaja de `SAVEPOINT` es permitir un rollback parcial y mas controlado.

Ejercicio 6. Concurrency y Bloqueos

Objetivo

Simular un bloqueo entre dos sesiones que intentan modificar el mismo registro.

Script Propuesto

```
-- SESION 1  
BEGIN TRAN;  
UPDATE dbo.Empleado  
SET Empleado_SueldoBase = 83000.00  
WHERE Empleado_ID = 3;  
-- Mantener abierta la transaccion.
```

```
-- SESION 2  
UPDATE dbo.Empleado  
SET Empleado_SueldoBase = 84000.00  
WHERE Empleado_ID = 3;
```

Explicacion Detallada

En la primera sesion, SQL Server obtiene un bloqueo exclusivo sobre la fila del empleado 3. Mientras esa transaccion no haga `COMMIT` o `ROLLBACK`, la segunda sesion no puede modificar la misma fila. Por eso queda en espera.

Este comportamiento protege la consistencia. Si ambos usuarios pudieran sobrescribir al mismo tiempo el mismo registro sin coordinacion, se producirian inconsistencias o perdida de cambios.

Respuestas

1. En la segunda sesion la sentencia queda bloqueada o esperando el recurso.
2. El bloqueo ocurre porque la primera transaccion retiene un bloqueo exclusivo sobre la fila.

Ejercicio 7. Niveles de Aislamiento

Objetivo

Aplicar `READ COMMITTED` y explicar el problema que evita.

Script Ejecutado

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
SELECT *  
FROM dbo.TransaccionNomina;
```

Explicacion Detallada

Con `READ COMMITTED`, una consulta solo puede leer datos que ya hayan sido confirmados por otras transacciones. Esto significa que si una transaccion modifica un registro pero todavia no hace `COMMIT`, las demas no deberian leer ese valor intermedio.

El objetivo principal es evitar las llamadas `dirty reads`, o lecturas sucias. Ese problema ocurre cuando una transaccion lee un valor que despues es revertido y, por tanto, nunca debio considerarse valido.

Respuestas

1. Este nivel de aislamiento evita las lecturas sucias.
2. Una dirty read es la lectura de datos no confirmados que luego pueden ser revertidos.

Ejercicio 8. Stored Procedures

Objetivo

Crear y utilizar cinco procedimientos almacenados para el sistema de nomina.

Stored Procedures Implementados

```
SP_Actualizar_TransaccionesNomina  
SP_Consultar_NominaEmpleado  
SP_Insertar_EmpleadoLog  
SP_Insertar_TransaccionNominaLog  
SP_Insertar_TransaccionesNominaHistorico
```

Explicacion Detallada de Cada Procedimiento

`SP\_Actualizar\_TransaccionesNomina` toma una transaccion de nomina y recalcula sus montos financieros. Busca el sueldo base del empleado, calcula AFP, ARS e ISR, y luego actualiza el sueldo neto de la transaccion.

`SP\_Consultar\_NominaEmpleado` devuelve todas las transacciones asociadas a un empleado. Este procedimiento fue ademas reutilizado por un trigger para poblar una tabla cache.

`SP\_Insertar\_EmpleadoLog` registra operaciones realizadas sobre la tabla `Empleado`, como inserciones, actualizaciones o eliminaciones.

`SP\_Insertar\_TransaccionNominaLog` guarda la trazabilidad de cambios sobre `TransaccionNomina`.

`SP\_Insertar\_TransaccionesNominaHistorico` copia a una tabla historica la informacion de una transaccion eliminada.

Ejemplo de Ejecucion

```
EXEC dbo.SP_Actualizar_TransaccionesNomina
    @TransaccionNomina_ID = 3,
    @Compania_Codigo = 'UAS',
    @Empleado_Codigo = 'E00003',
    @SueldoBruto = 78000.00;

EXEC dbo.SP_Consultar_NominaEmpleado
    @Empleado_ID = 3;
```

Resultado Esperado de la Consulta del Empleado

Empleado_ID	Transaccion_ID	Numero	Empleado	Periodo	Monto Bruto	Monto Neto
3	3	1003	Victor Perez Padilla	202601	78000.00	66459.71
3	6	1006	Victor Perez Padilla	202602	80000.00	67871.06

Respuestas

1. Las ventajas principales de los stored procedures son reutilizacion, mantenimiento centralizado, seguridad y consistencia en la logica del negocio.
2. Se ejecutan con `EXEC`.

Ejercicio 9. Funciones en SQL Server

Objetivo

Crear una funcion que calcule el salario anual de un empleado a partir de su sueldo mensual.

Script Ejecutado

```
CREATE FUNCTION dbo.FN_SalarioAnual
(
    @SueldoMensual DECIMAL(12,2)
)
RETURNS DECIMAL(14,2)
AS
BEGIN
    RETURN ROUND(ISNULL(@SueldoMensual, 0.00) * 12.00, 2);
END;
```

```
SELECT
    E.Empleado_Codigo,
    E.Empleado_Nombre + ' ' + E.Empleado_Apellido AS Empleado,
    E.Empleado_SueldoBase AS SueldoMensual,
    dbo.FN_SalarioAnual(E.Empleado_SueldoBase) AS SalarioAnual
FROM dbo.Empleado AS E;
```

Explicacion Detallada

La funcion recibe un sueldo mensual y devuelve el equivalente anual. Aunque el calculo es sencillo, sirve para demostrar encapsulamiento de logica reutilizable. En vez de repetir `sueldo \* 12` en todas las consultas, la regla queda concentrada en una funcion.

Resultado Esperado de la Corrida

Codigo	Empleado	Sueldo Mensual	Salario Anual
E00001	Marlenis Judith Concepcion Cuevas	42000.00	504000.00
E00002	Roberto Byas De la Cruz	55000.00	660000.00
E00003	Victor Perez Padilla	78000.00	936000.00
E00004	Manicaotex Mueses	120000.00	1440000.00
E00005	Rosmery Alberto Martinez	250000.00	3000000.00
E00006	Ana Martinez	32000.00	384000.00
E00007	Jose Amado	68000.00	816000.00

Respuesta

Las funciones son recomendables cuando se necesita una logica reutilizable, consistente y facil de invocar desde multiples consultas.

Ejercicio 10. Triggers

Objetivo

Crear cinco triggers asociados a los procedimientos almacenados del sistema.

Triggers Implementados

```
TR_TransaccionNomina_AfterInsert_Actualizar  
TR_TransaccionNomina_AfterInsert_ConsultarEmpleado  
TR_Empleado_Audit  
TR_TransaccionNomina_Audit  
TR_TransaccionNomina_Historico
```

Explicacion Detallada de Cada Trigger

`TR\_TransaccionNomina\_AfterInsert\_Actualizar` se ejecuta despues de insertar una transaccion y llama al procedimiento que calcula AFP, ARS, ISR y sueldo neto.

`TR\_TransaccionNomina\_AfterInsert\_ConsultarEmpleado` se ejecuta al insertar y llena una tabla cache con el resultado de `SP\_Consultar\_NominaEmpleado`.

`TR\_Empleado\_Audit` registra auditoria cuando la tabla `Empleado` recibe `INSERT`, `UPDATE` o `DELETE`.

`TR\_TransaccionNomina\_Audit` guarda la trazabilidad de inserciones, modificaciones y eliminaciones sobre transacciones de nomina.

`TR\_TransaccionNomina\_Historico` archiva en historico una transaccion eliminada.

Evidencias Esperadas

EmpleadoLog_ID	Empleado_ID	Codigo	NombreCompleto	Accion
2	7	E00007	Jose Amado	IN
SERT				
1	1	E00001	Marlenis Judith Concepcion Cuevas	UP
DATE				

Cache_ID	Empleado_ID	Numero	Empleado_Codigo	Periodo	Monto Bruto	Monto Neto
1	7	1010	E00007	202603	68000.00	58989.11

TransaccionLog_ID	Numero	Empleado_ID	Monto Bruto	Monto Neto	Accion
12	1010	7	68000.00	58989.11	DELETE
11	1010	7	68000.00	58989.11	UPDATE

Historico_ID	Numero	Empleado_ID	Monto Bruto	Monto Neto	Operacion
1	1010	7	68000.00	58989.11	DELETE

Respuesta

La principal ventaja de los triggers es automatizar tareas de auditoria, integridad e historico sin depender de que el usuario ejecute procesos manuales.

Ejercicio 11. Vista Materializada

Objetivo

Crear una vista que muestre el total pagado a cada empleado.

Script Ejecutado

```
CREATE VIEW dbo.VW_TotalNominaEmpleado
WITH SCHEMABINDING
AS
    SELECT
        T.Empleado_ID,
        COUNT_BIG(*) AS CantidadTransacciones,
        SUM(ISNULL(T.TransaccionNomina_SuelDonNeto, 0.00)) AS TotalPagado
    FROM dbo.TransaccionNomina AS T
    WHERE T.TransaccionNomina_Estado = 'Aplicada'
    GROUP BY T.Empleado_ID;

CREATE UNIQUE CLUSTERED INDEX IX_VW_TotalNominaEmpleado
ON dbo.VW_TotalNominaEmpleado (Empleado_ID);
```

```
SELECT
    V.Empleado_ID,
    E.Empleado_Codigo,
    E.Empleado_Nombre + ' ' + E.Empleado_Apellido AS Empleado,
    V.CantidadTransacciones,
    V.TotalPagado
FROM dbo.VW_TotalNominaEmpleado AS V
INNER JOIN dbo.Empleado AS E
    ON E.Empleado_ID = V.Empleado_ID
ORDER BY V.TotalPagado DESC;
```

Explicacion Detallada

En SQL Server, una vista materializada se implementa practicamente mediante una `indexed view`. Primero se crea la vista con `WITH SCHEMABINDING`, lo que impide cambios

estructurales en las tablas base que rompan la definicion. Luego se crea un indice clustered unico sobre la vista para materializar su estructura.

La consulta final muestra cuantas transacciones aplicadas tiene cada empleado y cuanto ha recibido en total como sueldo neto.

Resultado Esperado de la Corrida

Empleado_ID	Codigo	Empleado	Cantidad	Total Paga
5	E00005	Rosmery Alberto Martinez	1	188240.97
3	E00003	Victor Perez Padilla	2	134330.77
2	E00002	Roberto Byas De la Cruz	2	98379.66
4	E00004	Manicaotex Mueses	1	96098.06
1	E00001	Marlenis Judith Concepcion Cuevas	2	77585.76
6	E00006	Ana Martinez	1	30108.80

Respuestas

1. La diferencia es que la vista normal es virtual, mientras que la vista materializada o indexada mantiene estructura fisica para acelerar consultas.
2. Se recomienda cuando hay consultas agregadas frecuentes, con mucha lectura y necesidad de mejorar rendimiento.

Ejercicio 12. Seguridad: Usuarios, Roles y Permisos

Objetivo

Crear un login, un usuario y un rol de solo lectura para el area de nomina.

Script Ejecutado

```
USE master;

CREATE LOGIN login_consulta_nomina_marlenis
WITH PASSWORD = 'Nomina#UASD2026';

USE DBNomina;

CREATE USER user_consulta_nomina_marlenis
FOR LOGIN login_consulta_nomina_marlenis;

CREATE ROLE RolConsultaNomina;

GRANT SELECT ON dbo.Compania TO RolConsultaNomina;
GRANT SELECT ON dbo.Departamento TO RolConsultaNomina;
GRANT SELECT ON dbo.Empleado TO RolConsultaNomina;
GRANT SELECT ON dbo.TipoNomina TO RolConsultaNomina;
GRANT SELECT ON dbo.PeriodoNomina TO RolConsultaNomina;
GRANT SELECT ON dbo.TransaccionNomina TO RolConsultaNomina;
GRANT SELECT ON dbo.VW_TotalNominaEmpleado TO RolConsultaNomina;

ALTER ROLE RolConsultaNomina
ADD MEMBER user_consulta_nomina_marlenis;
```

Explicacion Detallada

Primero se crea un login a nivel de servidor. Luego se crea un usuario dentro de la base `DBNomina` asociado a ese login. Despues se define el rol `RolConsultaNomina`, al cual se le otorgan permisos `SELECT` sobre las tablas y vista necesarias. Finalmente, el usuario queda agregado al rol.

Con este enfoque, el usuario puede consultar informacion de nomina, pero no modificarla. Esto representa una aplicacion correcta del principio de minimo privilegio.

Resultado Esperado

```
Login creado correctamente.  
Usuario de base de datos creado correctamente.  
Rol RolConsultaNomina creado correctamente.  
Permisos SELECT otorgados correctamente.  
Usuario agregado al rol correctamente.
```

Respuestas

1. El principio de minimo privilegio significa otorgar solamente los permisos necesarios para cumplir una funcion.
2. La seguridad en bases de datos es importante para proteger confidencialidad, integridad, disponibilidad y trazabilidad.

Conclusiones

La actividad permitio integrar varias areas avanzadas de SQL Server en una sola solucion. No se trabajo solamente el diseno de tablas, sino tambien la forma en que el motor ejecuta consultas, optimiza acceso mediante indices, administra transacciones y automatiza procesos a traves de procedimientos, funciones y triggers.

El resultado final fue una base `DBNomina` mas completa que la planteada originalmente en el enunciado, porque ademas de resolver los ejercicios, incorpora elementos reales de auditoria, historico, cache y seguridad. Esto hace que la solucion sea mas cercana a un escenario profesional y no solamente academico.

Referencias

Dirección General de Impuestos Internos. (s. f.). Impuesto sobre la renta. Recuperado el 19 de abril de 2026, de

<https://dgii.gov.do/cicloContribuyente/obligacionesTributarias/principalesImpuestos/Paginas/impuestoSobr>

Dirección General de Impuestos Internos. (2026, 16 de enero). CA687: Cual es la escala salarial correspondiente al año 2026 del impuesto sobre la renta (ISR)? Comunidad de Ayuda de la Dirección General de Impuestos Internos.

<https://ayuda.dgii.gov.do/conversations/impuesto-sobre-la-renta-isr/ca687-cul-es-la-escala-salarial-correspo>

Tesorería de la Seguridad Social. (2024, julio). Manual de preguntas frecuentes (version 2.0).

<https://tss.gob.do/descargar/506/archivos-variados-sueltos/5332/faq0725-2024.pdf>

Tesorería de la Seguridad Social. (2025, 4 de abril). Tesorería de la Seguridad Social informa nuevos topes de cotización para el régimen contributivo.

<https://tss.gob.do/tesoreria-de-la-seguridad-social-informa-nuevos-topes-de-cotizacion-para-el-regimen-con>